MULUNGUSHI UNIVERSITY
School of Engineering and Technology

ICS 441

Advanced Cybersecurity and Ethical Hacking

Unit 1: Engagement Ethics, Law, Safety, Threat Modeling and Risk Register

Lecture 1 • Lecturer: Mr. M Bwalya



Core rule: no written authorisation, no testing.

Unit learning outcomes

By the end of this unit, students should be able to:

- Separate ethical hacking from cybercrime using authorization, scope and intent.
- Prepare a safe engagement plan with Rules of Engagement and written consent.
- Handle evidence and discovered data using privacy and OPSEC principles.
- Explain threat modeling and convert threats into safe security test cases.
- Create a simple risk register with likelihood, impact, mitigation, owner and status.

Professional security work is planned, legal, safe and documented.

Course mindset: red team thinking, blue team responsibility

Red-team thinking

- Ask how a weakness could be discovered.
- Validate risk only in authorised safe-lab conditions.
- Document what happened clearly and honestly.
- Think creatively, but never act outside the ROE.

Blue-team responsibility

- Prevent weaknesses through secure design and configuration.
- Detect suspicious behaviour using logs and analytics.
- Respond quickly and contain incidents.
- Recover and improve controls after lessons learned.

What makes this course advanced?

Beyond basic awareness

- Move from “what is malware?” to how attacks progress across systems.
- Connect identity, networks, endpoints, applications, data and cloud.
- Study both offensive testing and defensive detection.
- Treat security as risk management, not just tool use.

Professional capability

- Use frameworks such as MITRE ATT&CK, OWASP and NIST.
- Collect evidence through logs, scans and lab observations.
- Write findings for technical and non-technical audiences.
- Prioritise remediation using business impact.

The modern attack surface

Traditional surface

- Networks, servers and endpoints.
- Open ports, weak patching and poor segmentation.
- User laptops, desktops and mobile devices.

Modern surface

- Cloud environments, web apps and APIs.
- Containers, CI/CD pipelines and supply chains.
- IoT, edge devices and third-party integrations.

Teaching focus

- Understand where systems are exposed.
- Test only within approved lab scope.
- Map weaknesses to controls and evidence.

Ethical hacking in context



Visual summary: authorisation, ROE, privacy, evidence, audit trail and responsible disclosure.

Ethical hacking versus cybercrime

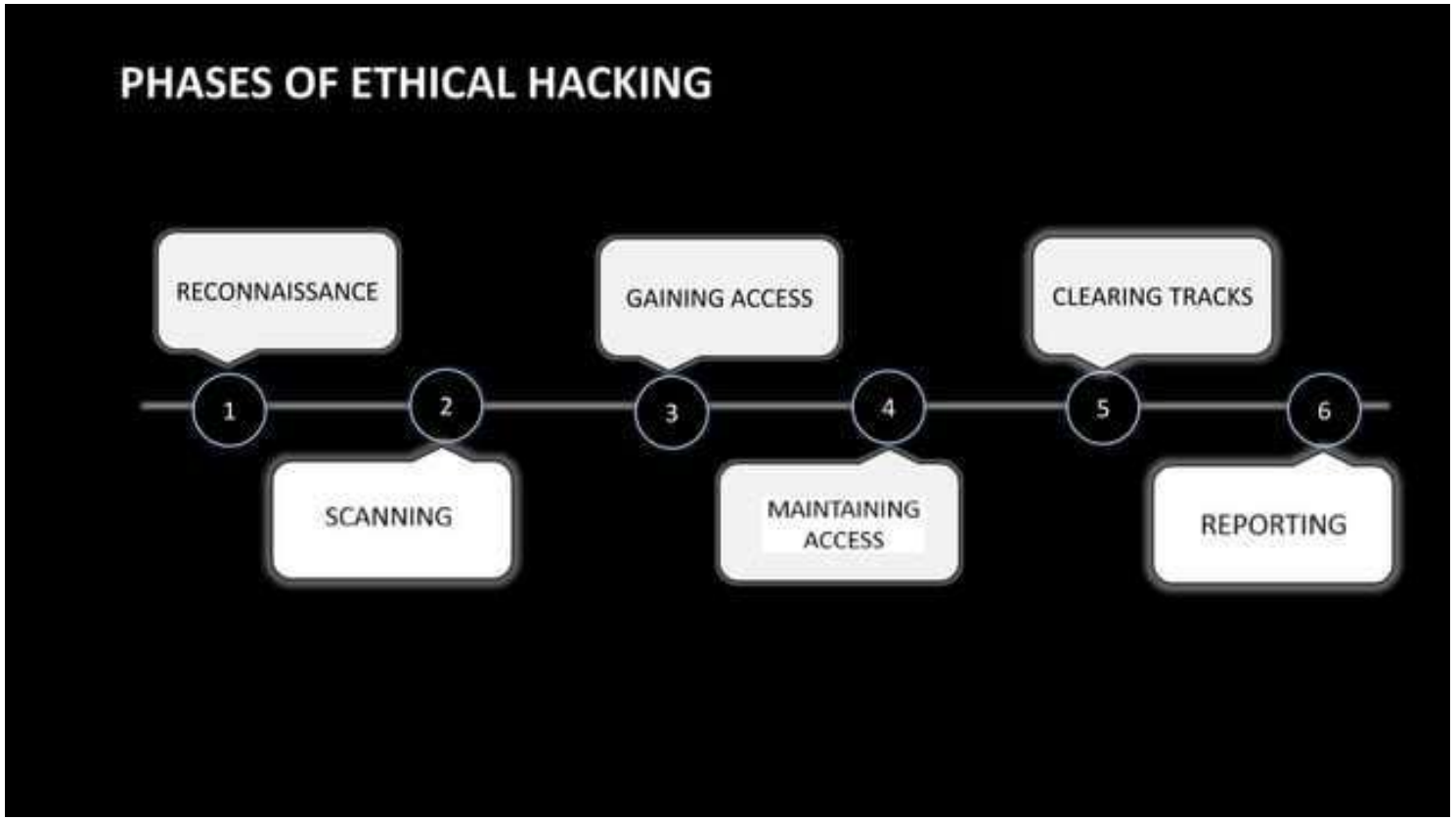
Ethical hacking

- Legal and explicitly authorised by the system owner.
- Has written scope, approved methods and a defined time window.
- Intent is to improve security and reduce risk.
- Findings are documented and reported responsibly.

Cybercrime

- No permission from the owner.
- May involve theft, disruption, data exposure or coercion.
- Intent is harmful, selfish or unlawful.
- Creates legal, disciplinary and reputational consequences.

Phases of ethical hacking — interpreted safely



Safe ethical-hacking workflow

From permission to improvement



Authorize	Plan	Observe	Validate	Report	Retest
Written consent and signed scope.	ROE, timeline, contacts and limits.	Reconnaissance within permitted scope.	Test safely without unnecessary harm.	Evidence, impact and remediation.	Confirm fix and close findings.

Every step must remain authorised, scoped, logged and reversible.

Key frameworks we will use

MITRE ATTandCK

- Common language for adversary tactics and techniques.
- Useful for mapping tests to expected detections.
- Helps identify coverage gaps.

OWASP

- Source for web application security testing.
- Supports learning on auth, sessions, APIs and secure coding.
- Provides practical checklists and testing guidance.

NIST guidance

- Supports testing, incident handling and risk management.
- Useful for structured professional documentation.
- Connects technical findings to governance.

Defender response to offensive actions

Prevent

- Secure configuration and hardening.
- Least privilege and MFA.
- Secure coding and patching.

Detect

- Logs, SIEM, IDS and analytics.
- Alert on suspicious activity.
- Measure visibility across systems.

Respond and recover

- Triage and contain incidents.
- Restore services and preserve evidence.
- Document lessons learned.

Engagement ethics: why safety comes first

Important teaching points

- Technical skills are irrelevant without a legal and ethical framework.
- Authorization is the line between a security professional and a criminal.
- Testing must minimize disruption, privacy harm and data exposure.
- A professional tester can justify every action taken during the engagement.
- When risk is unclear, pause and escalate instead of improvising.

No permission. No scope. No testing.

Rules of Engagement (ROE): the safety contract

What ROE defines

- In-scope and out-of-scope assets.
- Allowed methods, tools, techniques and rate limits.
- Testing window, maintenance windows and stop conditions.
- Contacts, escalation paths and response service levels.

Why ROE matters

- Prevents accidental harm to production services.
- Protects students, staff and the institution.
- Makes testing auditable and defensible.
- Turns vague permission into operational boundaries.

Defining scope clearly

In-scope examples

- Named lab hosts, subnets, URLs and APIs.
- Dedicated test accounts and sample data.
- Approved cloud sandbox resources.
- Specific time window and controlled test methods.

Out-of-scope examples

- Production databases and personal email.
- DoS, destructive actions or social engineering unless approved.
- Third-party systems not owned by the client.
- Any asset not explicitly named in the ROE.

Written consent before testing

What consent must include

- Exact systems, accounts, URLs or networks approved for testing.
- Approved dates, times, techniques and emergency stop conditions.
- Contacts for escalation and rapid reporting of critical findings.
- Data-handling, non-disclosure and evidence storage rules.
- Signatures from the authorised owner or delegated authority.

Store consent with the test plan; update it when scope changes.

Responsible disclosure of vulnerabilities

Internal classroom process

- Report promptly with clear reproduction steps.
- Include screenshots, timestamps and affected assets where needed.
- Propose severity and feasible mitigation.
- Coordinate retesting after the owner fixes the issue.

Quality expectations

- Be factual, neutral and respectful.
- Avoid publishing details without approval.
- Do not include unnecessary exploit code.
- Focus on helping the owner reduce risk.

Operational Security (OPSEC) in the lab

Network and host hygiene

- Use isolated networks such as host-only or NAT.
- Create VM snapshots before risky activities.
- Label test machines clearly.
- Block unintended outbound traffic when required.

Credential safety

- Use dedicated test accounts.
- Never reuse personal or production credentials.
- Rotate passwords after exercises.

Cybersecurity hygiene reminders



Source visual from the attached lecture notes; use it as a classroom discussion prompt.

Secure handling and privacy of discovered data

Data minimisation

- Collect only what is necessary to prove the finding.
- Avoid bulk downloads of sensitive data.
- Stop and escalate if unexpected personal data appears.

Evidence protection

- Classify evidence by sensitivity.
- Redact names, IDs, passwords and tokens.
- Encrypt evidence at rest.
- Restrict access using least privilege.

Evidence management and audit trail

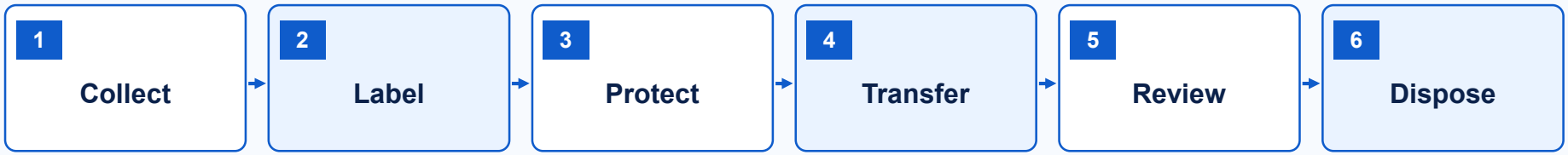
Good evidence practices

- Record timestamps, commands, hashes, source IPs and observed results.
- Use screenshots only when they add clarity and crop them safely.
- Caption screenshots so they match log lines or test steps.
- Store evidence in approved locations, not personal cloud-sync folders.
- Define retention and secure disposal rules in the engagement plan.

Evidence should be accurate, minimal, protected and traceable.

Chain of custody for sensitive evidence

Protecting trust in evidence



Capture minimum proof.

Asset, source and timestamp.

Encrypt and restrict access.

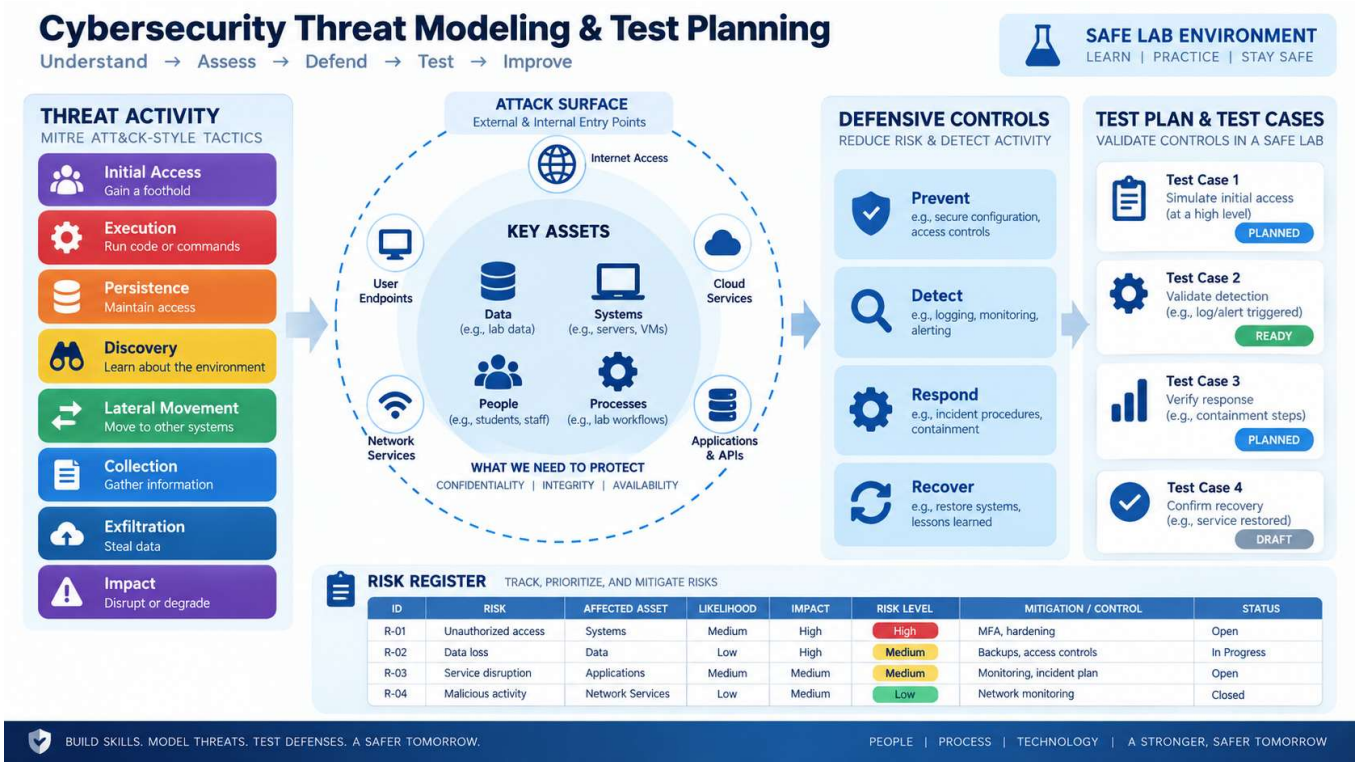
Record who received it.

Use for remediation.

Securely delete after sign-off.

Every step must remain authorised, scoped, logged and reversible.

Threat modeling and test planning overview



Identify assets, model the attack surface, select controls, create tests and track risk.

Threat modeling: definition and purpose

What is threat modeling?

- A structured process for identifying, quantifying and addressing security risks.
- It asks “what can go wrong?” before attackers or failures prove it in production.
- It helps teams understand the system attack surface and prioritize security effort.
- It is proactive: design weaknesses are cheaper to fix before implementation.
- It turns security thinking into diagrams, hypotheses, tests and controls.

Threat modeling moves security left.

Benefits of threat modeling

For security teams

- Finds design flaws early in the SDLC.
- Provides systematic coverage instead of random testing.
- Reduces the chance of missing critical threats.
- Generates meaningful security test cases.

For project teams

- Improves communication among developers, architects and testers.
- Creates shared understanding of security requirements.
- Supports prioritization when time and budget are limited.
- Makes remediation easier to justify.

Threat modeling frameworks

STRIDE

- Spoofing, Tampering, Repudiation.
- Information Disclosure, Denial of Service.
- Elevation of Privilege.

PASTA and DREAD

- PASTA is risk-centric and attack-simulation oriented.
- DREAD estimates damage, reproducibility, exploitability, affected users and discoverability.

OWASP Threat Modeling Guide

- Practical guidance for application threat modeling.
- Useful for turning design diagrams into security questions.

Security test planning

What is a security test plan?

- A document or strategy that defines scope, objectives, resources, schedule and methods.
- It translates threat-model output into actionable test cases.
- It states what evidence will prove whether a control works.
- It defines safety controls before testing starts.

Common approaches

- SAST: analyze source code for vulnerabilities.
- DAST: test a running application for vulnerabilities.
- Penetration testing: simulate a controlled real-world attack.
- Review and retest: confirm remediation effectiveness.

The crucial link: threats become test cases

How threat modeling informs testing

- The primary threat-model output is a list of potential threats and attack vectors.
- Each threat should become a safe, scoped and repeatable test case.
- Each test case needs an objective, steps, expected evidence and success criteria.
- Tests should map to defensive controls such as logs, alerts and access policies.
- This makes assessment systematic rather than guesswork.

Model → Test → Evidence → Mitigation → Retest

Identifying key assets and crown jewels

Crown jewels

- Sensitive data such as student, financial and health records.
- Critical services such as authentication, billing, payroll and registration.
- Brand reputation, intellectual property, API keys and source code.
- Business processes that must not fail.

System inventory

- Web applications, APIs, endpoints and servers.
- Active Directory, cloud resources, VMs and databases.
- Third-party dependencies and integrations.
- Public repositories, auth providers and payment services.

Attack surface analysis

What is the attack surface?

- All possible points where an attacker could enter, influence or extract data.
- Network interfaces, open ports and exposed services.
- Web applications, APIs, mobile apps and user endpoints.
- Cloud storage, human factors and third-party systems.

Entry points and exposures

- Login and password reset forms.
- File upload, admin panels and public buckets.
- Default credentials and directory listing.
- Misconfigured services and unnecessary open ports.

Using MITRE ATTandCK in test planning

Core idea

- Tactics describe attacker goals such as initial access or exfiltration.
- Techniques describe how attackers achieve those goals.
- ATTandCK gives red and blue teams a shared language.

How students use it

- Tag each hypothesis with a tactic or technique.
- List expected HTTP, authentication, database or cloud logs.
- Identify detection gaps and control weaknesses.
- Prioritize tests tied to weak or missing detections.

Threat hypotheses and safe test cases

Threat hypothesis format

- State how an attacker might compromise a system.
- Name the affected asset and likely attack path.
- Example: weak IAM policies may expose sensitive cloud storage.
- Connect the hypothesis to business impact.

Test case format

- Objective: what exactly is being tested?
- Steps: safe, scoped, repeatable actions.
- Success criteria: what result proves the issue?
- Outcome: evidence and recommendation recorded.

Creating a Basic Risk Register

• **Purpose:** To document, prioritize, and track security risks.
Key Components:

- **Risk Description:** What is the vulnerability or threat?
- **Likelihood:** How probable is it that this will be exploited? (e.g., Low, Medium, High)
- **Impact:** What would be the business effect if exploited? (e.g., Financial loss, reputational damage)
- **Risk Score:** Likelihood × Impact (used for prioritization).
- **Mitigation Steps:** Recommended actions to reduce risk.
- **Owner:** Who is responsible for addressing this?

Basic risk register: example format

ID	Risk / Hypothesis	Asset	Likelihood	Impact	Mitigation / Control	Status
R-01	Weak MFA on admin accounts	Identity	High	High	Enforce MFA, review roles	Open
R-02	Public file upload exposure	Web app	Medium	High	Restrict file types, scan uploads	In progress
R-03	Default service banner leaks info	API	Medium	Medium	Harden headers and minimize banners	Open
R-04	Unencrypted evidence storage	Evidence	Low	High	Encrypt evidence folder and restrict access	Mitigated
R-05	Unpatched lab VM image	Endpoint	Medium	Medium	Patch template and retest	Open

Risk register = track, prioritize, assign ownership, mitigate and retest.

Mini-case discussion: safe lab incident

Scenario and discussion questions

- A student accidentally scans outside the approved lab subnet.
- Monitoring alarms trigger and the network team asks what happened.
- Which ROE controls should have prevented the mistake?
- What should the student do immediately?
- What evidence should be preserved and who should be informed?

Safety failures are process failures; improve the process, not only the person.

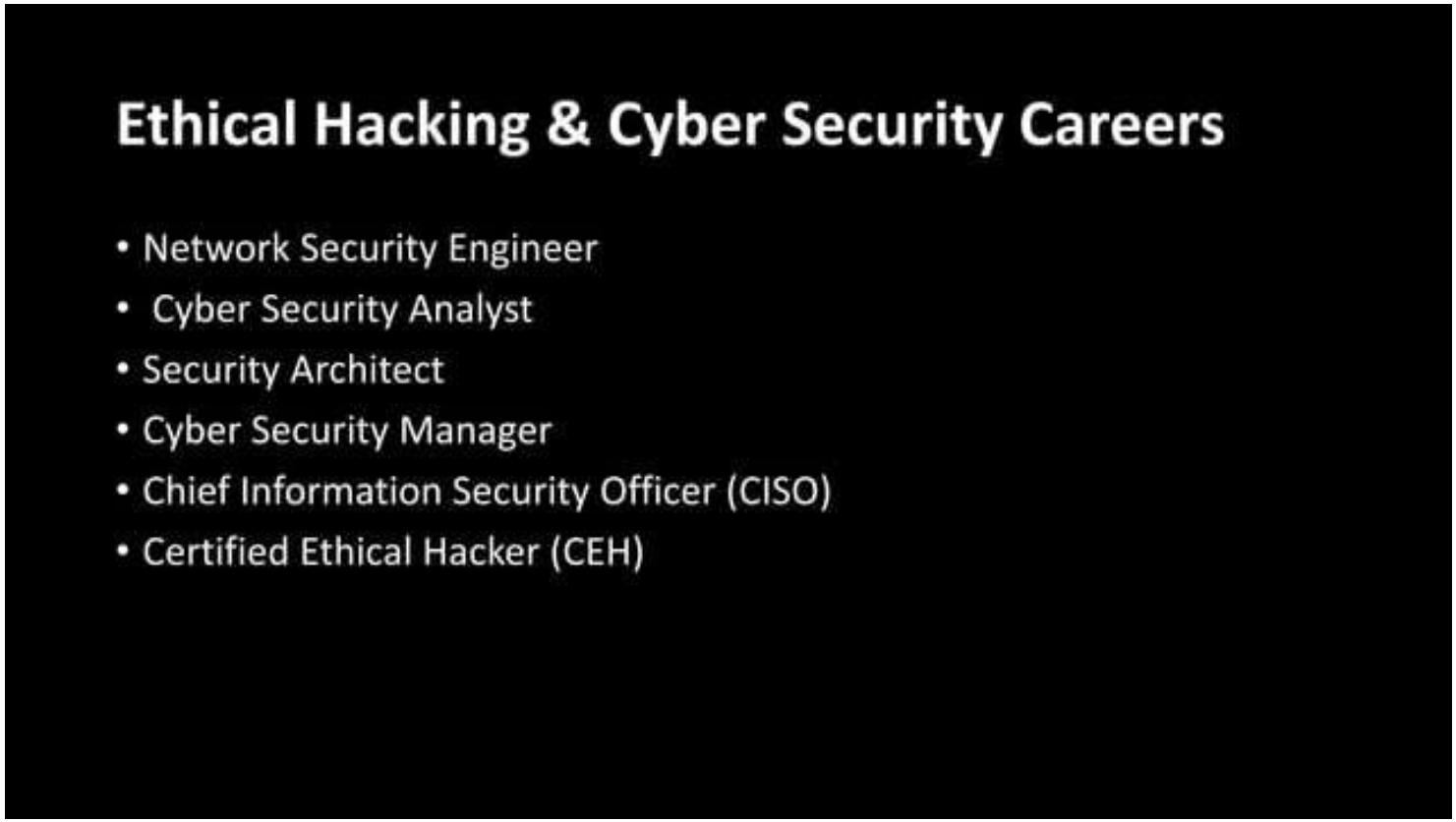
Quick knowledge check

Answer in pairs

- What is the difference between ethical hacking and cybercrime?
- Name four items that should appear in a Rules of Engagement document.
- Why is data minimization important during security testing?
- How does a threat hypothesis become a test case?
- What columns belong in a basic risk register?
- Why must evidence be protected and traceable?

Use examples, not memorized definitions only.

Careers linked to this unit



Key takeaways

Remember these points

- Ethical hacking is authorized, scoped, documented and risk-reducing.
- Rules of Engagement protect the tester, owner, institution and users.
- Threat modeling asks what can go wrong before it happens.
- A test plan converts threat hypotheses into safe, repeatable validation.
- A risk register tracks risk, likelihood, impact, mitigation, owner and status.

Skill + ethics + evidence + judgment.

Questions and next preparation

The aim is not to create reckless hackers. The aim is to develop security professionals who can analyze systems, understand risk, test safely and strengthen organizations.

Before the next class

THANK YOU